

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	7
1.2 Описание выходных данных.....	7
2 МЕТОД РЕШЕНИЯ.....	9
3 ОПИСАНИЕ АЛГОРИТМОВ.....	11
3.1 Алгоритм конструктора класса Sensor.....	11
3.2 Алгоритм метода setXY класса Sensor.....	11
3.3 Алгоритм метода getXY класса Sensor.....	12
3.4 Алгоритм метода setX класса Sensor.....	12
3.5 Алгоритм конструктора класса SmartSensor.....	12
3.6 Алгоритм метода setXY класса SmartSensor.....	13
3.7 Алгоритм метода getXY класса SmartSensor.....	13
3.8 Алгоритм функции main.....	14
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	16
5 КОД ПРОГРАММЫ.....	21
5.1 Файл main.cpp.....	21
5.2 Файл Sensor.cpp.....	22
5.3 Файл Sensor.h.....	22
5.4 Файл SmartSensor.cpp.....	23
5.5 Файл SmartSensor.h.....	24
6 ТЕСТИРОВАНИЕ.....	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26

1 ПОСТАНОВКА ЗАДАЧИ

В системе мониторинга используется **базовый датчик**, который хранит внутренний калибровочный параметр и измеряемое значение. На основе базового датчика разрабатывается **умный датчик**, который наследует свойства базового датчика, но имеет собственные поля и методы с теми же именами.

Требуется разработать систему, которая

- демонстрирует создание объекта производного класса;
- демонстрирует сохранение доступа к открытым элементам базового класса;
- демонстрирует сокрытие одноимённых элементов базового класса;
- демонстрирует однозначное обращение к элементам базового и производного классов.

Базовый класс должен содержать:

- в закрытом доступе:
 - одно свойство целого типа;
 - метод с одним параметром, который устанавливает значение закрытого свойства равным утроенному значению параметра.
- в открытом доступе:
 - одно свойство целого типа;
 - параметризованный конструктор с двумя параметрами(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
 - метод с двумя параметрами для изменения значений свойств(первый параметр для установки закрытого свойства через закрытый метод, второй для установки открытого свойства);
 - метод вывода значений свойств в формате "Sensor: <закрытое

свойство> <открытое свойство>"

Производный класс должен наследоваться от базового класса таким образом, чтобы к открытым элементам исходного класса сохранить открытый доступ и у него:

- в закрытом доступе:
 - о одно свойство целого типа с тем же именем, что и закрытое свойство базового класса.
- в открытом доступе:
 - о одно свойство целого типа с тем же именем, что и открытое свойство базового класса;
 - о параметризованный конструктор (1 параметр - установка закрытого свойства, 2 параметр - установка открытого свойства);
 - о метод изменения значений свойств с двумя параметрами;
 - о метод вывода значений свойств в формате: "SmartSensor: <закрытое свойство производного класса> <открытое свойство производного класса>"

Алгоритм работы программы

1. Объявить две целочисленные переменные.
2. Ввести значения двух переменных.
3. Создать объект производного класса.
4. Вывести значения свойств базовой части объекта.
5. Вывести значения свойств производной части объекта.
6. Ввести одно целое число.
7. Присвоить это значение открытому свойству производного класса.
8. Присвоить это значение открытому свойству базового класса.
9. Вывести значения свойств производного объекта.
10. Вывести значения свойств базового объекта.

11. Ввести два целых числа a и b .
12. Если первое число больше нуля:
 - 12.1. вызвать метод установки свойств производного объекта с параметрами $a+1$ и $b+1$;
 - 12.2. вызвать метод установки свойств базового объекта с параметрами $a-1$ и $b-1$;
 - 12.3. вывести значения производного объекта;
 - 12.4. вывести значения базового объекта.
13. Иначе:
 - 13.1. вызвать метод установки свойств базового объекта с параметрами $a+1$ и $b+1$;
 - 13.2. вызвать метод установки свойств производного объекта с параметрами $a-1$ и $b-1$;
 - 13.3. вывести значения базового объекта;
 - 13.4. вывести значения производного объекта.
14. Завершить работу программы.

1.1 Описание входных данных

Пример входных данных:

```
3 4
5
2 3
```

1.2 Описание выходных данных

Пример выходных данных:

```
Sensor: 9 4
SmartSensor: 3 4
SmartSensor: 3 5
```

Sensor: 9 5
SmartSensor: 3 4
Sensor: 3 2

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `ss` класса `SmartSensor` предназначен для описания объекта класса;
- объект `cin` класса `istream` предназначен для ввода значений с клавиатуры;
- объект `cout` класса `ostream` предназначен для вывода значения на экран;
- `if... else` - условный оператор.

Класс `Sensor`:

- свойства/поля:
 - поле целое число:
 - наименование — `x`;
 - тип — `int`;
 - модификатор доступа — `private`;
 - поле целое число:
 - наименование — `y`;
 - тип — `int`;
 - модификатор доступа — `public`;
- функционал:
 - метод `Sensor` — конструктор с параметрами;
 - метод `setXY` — метод для установления значений открытого и закрытого свойств;
 - метод `getXY` — метод для получения значений открытого и закрытого свойств;
 - метод `setX` — метод для получения утроенного значения

закрытого свойства.

Класс SmartSensor:

- свойства/поля:
 - о поле целое число:
 - наименование — x;
 - тип — int;
 - модификатор доступа — private;
 - о поле целое число:
 - наименование — y;
 - тип — int;
 - модификатор доступа — public;
- функционал:
 - о метод SmartSensor — конструктор с параметрами;
 - о метод setXY — метод для установления значений открытого и закрытого свойств;
 - о метод getXY — метод для получения значений открытого и закрытого свойств.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Sensor				
		SmartSensor	public		2
2	SmartSensor			производный класс	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса *Sensor*

Функционал: конструктор с параметрами.

Параметры: `int x`, `int y` - целые числа.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса *Sensor*

№	Предикат	Действия	№ перехода
1		применение метода <code>setX</code> с параметром <code>x</code>	2
2		присвоение значения <code>y</code> закрытому свойству <code>y</code>	Ø

3.2 Алгоритм метода `setXY` класса *Sensor*

Функционал: метод для установления значений открытого и закрытого свойств.

Параметры: `int x`, `int y` - целые числа.

Возвращаемое значение: `void`, ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода `setXY` класса *Sensor*

№	Предикат	Действия	№ перехода
1		применение метода <code>setX</code> с параметром <code>x</code>	2
2		присвоение значения <code>y</code> закрытому свойству <code>y</code>	Ø

3.3 Алгоритм метода getXY класса Sensor

Функционал: метод для получения значений открытого и закрытого свойств.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода getXY класса Sensor

№	Предикат	Действия	№ перехода
1		вывод на экран "Sensor: <закрытое свойства> <открытое свойство>"	Ø

3.4 Алгоритм метода setX класса Sensor

Функционал: метод для устроения значения закрытого свойства.

Параметры: int x - целое число.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода setX класса Sensor

№	Предикат	Действия	№ перехода
1		присвоение закрытому саойству x параметра x, умноженного на 3	Ø

3.5 Алгоритм конструктора класса SmartSensor

Функционал: конструктор с параметрами.

Параметры: int x, int y - целые числа .

Алгоритм конструктора представлен в таблице 6.

Таблица 6 – Алгоритм конструктора класса SmartSensor

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству x значения параметра x	2
2		присвоение открытому свойству y значения параметра y	∅

3.6 Алгоритм метода setXY класса SmartSensor

Функционал: метод для установления значений открытого и закрытого свойств.

Параметры: int x, int y - целые числа.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода setXY класса SmartSensor

№	Предикат	Действия	№ перехода
1		присвоение закрытому свойству x значения параметра x	2
2		присвоение открытому свойству y значения параметра y	∅

3.7 Алгоритм метода getXY класса SmartSensor

Функционал: метод для получения значений открытого и закрытого свойств.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода getXY класса SmartSensor

№	Предикат	Действия	№ перехода
1		вывод на экран "SmartSensor: <закрытое свойство производного класса> <открытое свойство производного класса>"	∅

3.8 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 9.

Таблица 9 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		объявление целых переменных x и y	2
2		ввод с клавиатуры переменных x и y	3
3		создание объекта ss класса SmartSensor с параметрами x и y	4
4		вызов метода getXY, принадлежащего базовому классу Sensor	5
5		вызов метода getXY	6
6		объявление целой переменной z	7
7		ввод с клавиатуры переменной z	8
8		присвоение значения z элементу базового класса (Sensor) y	9
9		присвоение значения z полю y объекта ss	10
10		вызов метода getXY	11
11		вызов метода getXY, принадлежащего базовому классу Sensor	12
12		объявление целых переменных a и b	13
13		ввод с клавиатуры переменных a и b	14
14	a>0	вызов метода setXY с параметрами a+1 и b+1	15
		вызов метода setXY, принадлежащего базовому классу Sensor	18

№	Предикат	Действия	№ перехода
		классу Sensor с параметрами a+1 и b+1	
15		вызов метода setXY, принадлежащего базовому классу Sensor с параметрами a-1 и b-1	16
16		вызов метода getXY	17
17		вызов метода getXY, принадлежащего базовому классу Sensor	∅
18		вызов метода setXY объекта ss с параметрами a-1 и b-1	19
19		вызов метода getXY, принадлежащего базовому классу Sensor	20
20		вызов метода getXY	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-5.

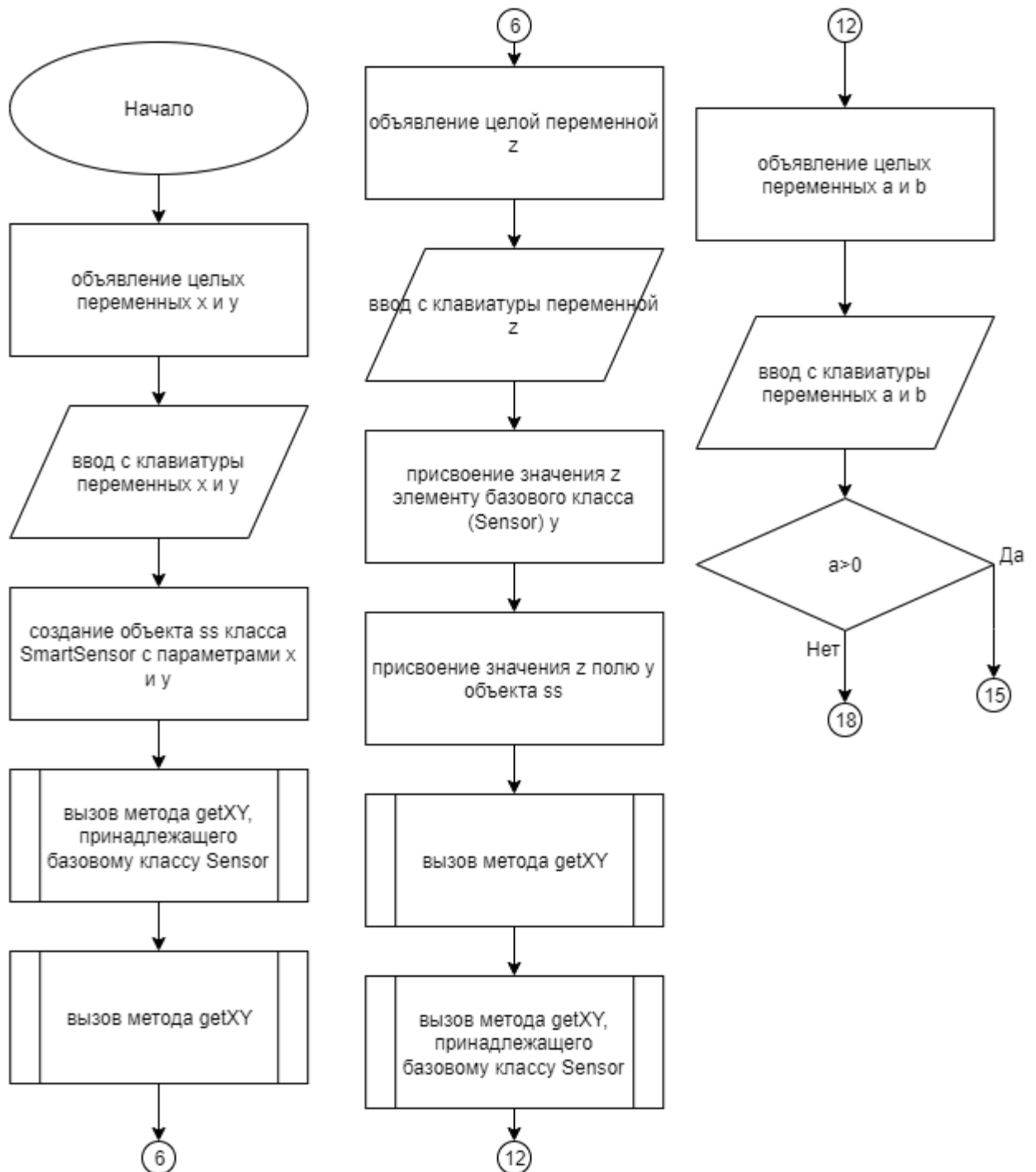


Рисунок 1 – Блок-схема алгоритма

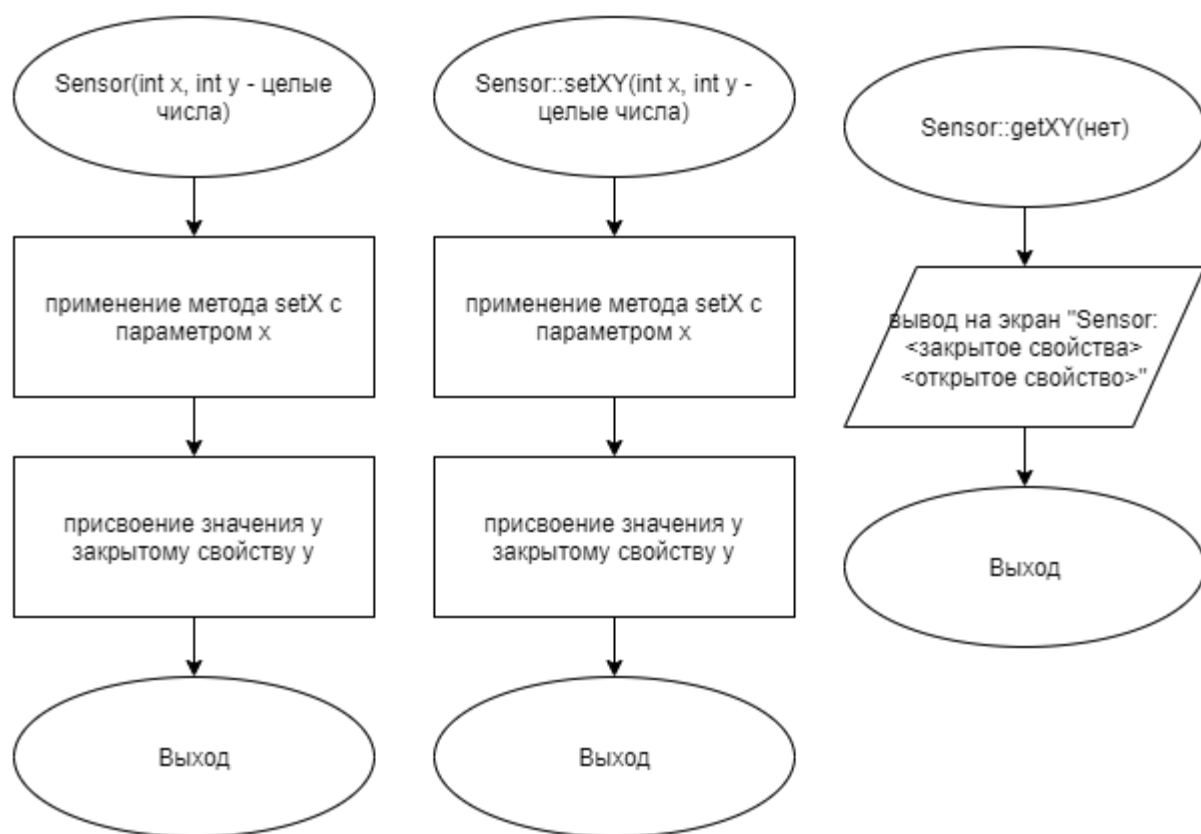


Рисунок 2 – Блок-схема алгоритма

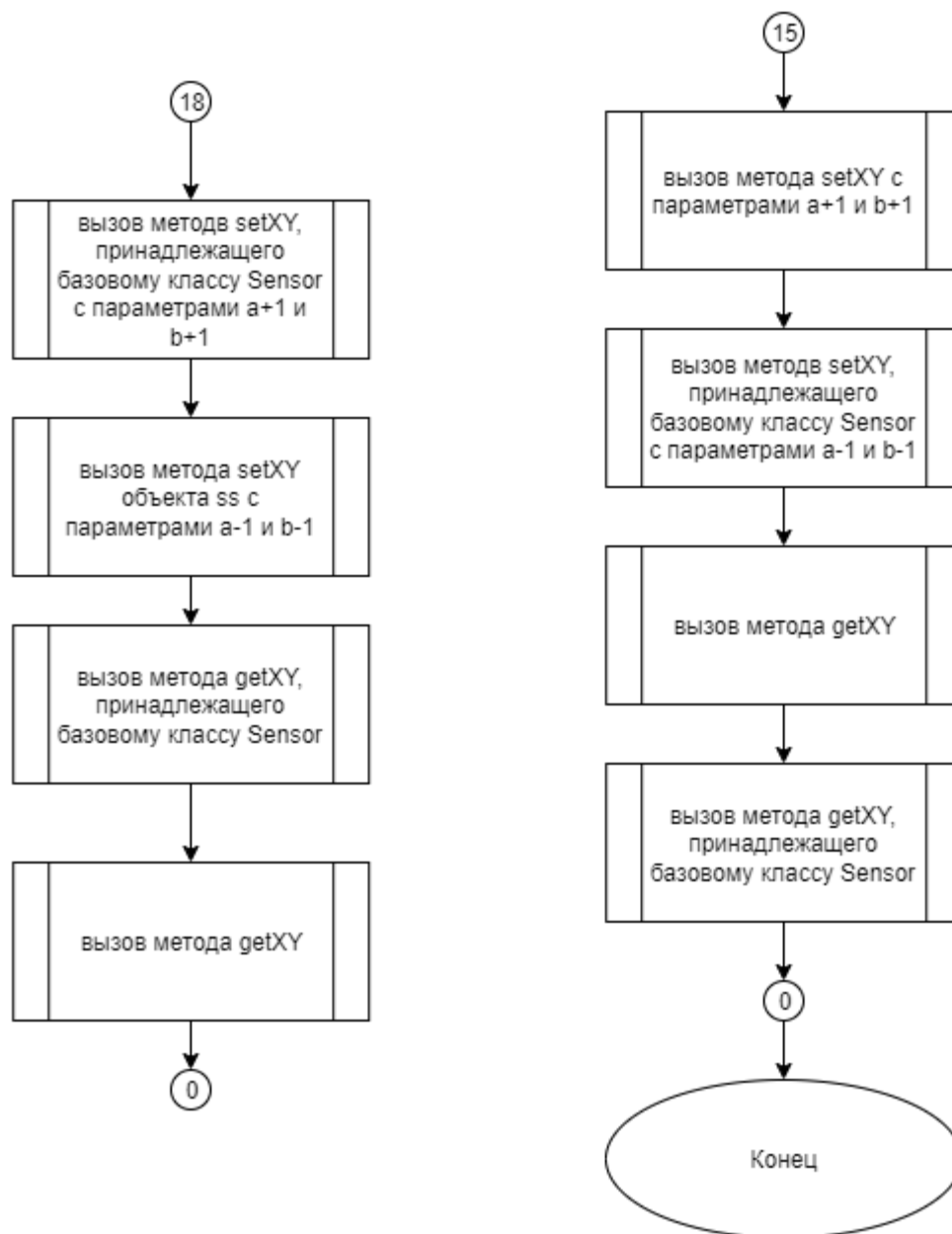


Рисунок 3 – Блок-схема алгоритма

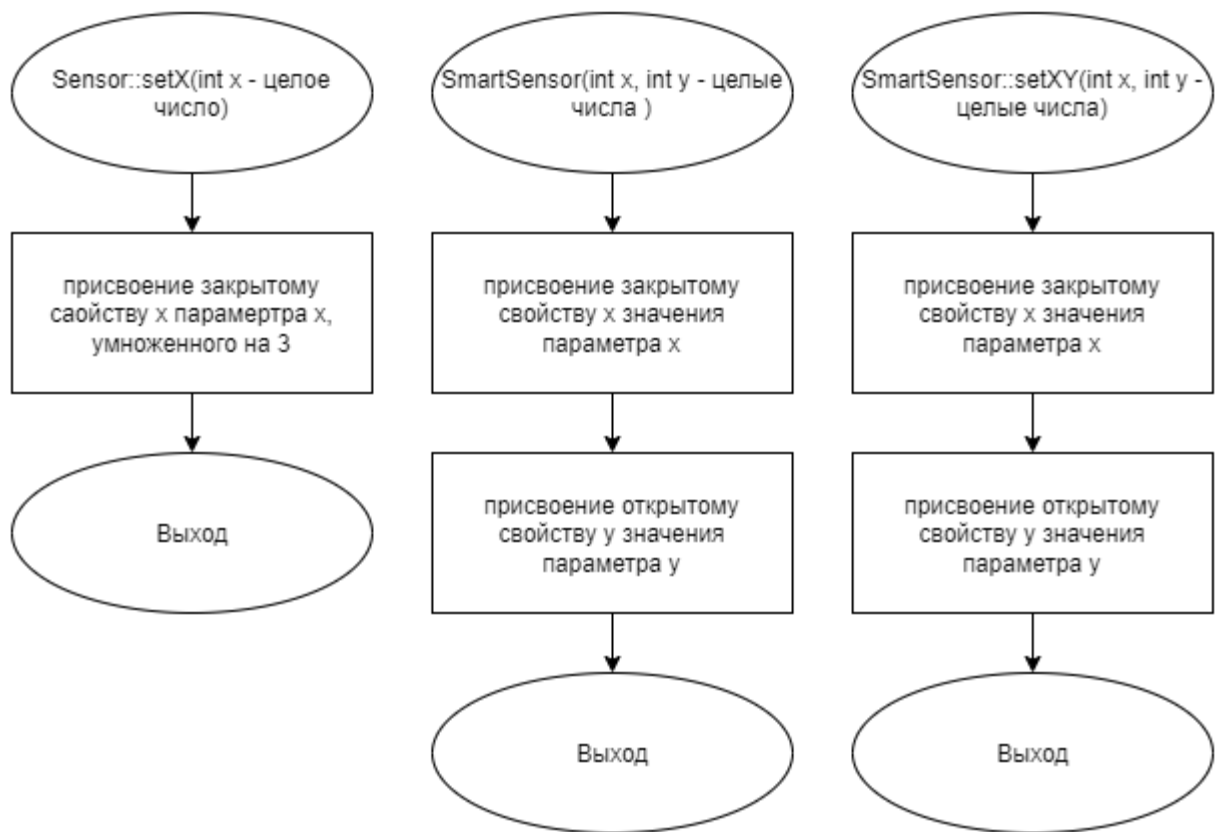


Рисунок 4 – Блок-схема алгоритма

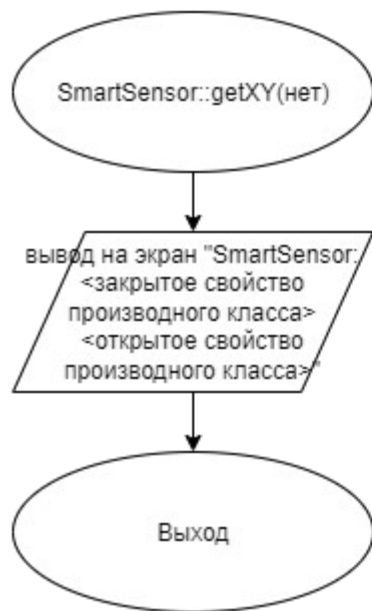


Рисунок 5 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл main.cpp

Листинг 1 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include "Sensor.h"
#include "SmartSensor.h"
#include <iostream>
using namespace std;

int main()
{
    int x, y;

    cin >> x >> y;

    SmartSensor ss(x, y);

    ss.Sensor::getXY();
    ss.getXY();

    int z;

    cin >> z;

    ss.Sensor::y = z;
    ss.y = z;

    ss.getXY();
    ss.Sensor::getXY();

    int a, b;

    cin >> a >> b;

    if (a>0)
    {
        ss.setXY(a+1, b+1);
        ss.Sensor::setXY(a-1, b-1);
        ss.getXY();
        ss.Sensor::getXY();
    }
}
```

```

        else
        {
            ss.Sensor::setXY(a+1, b+1);
            ss.setXY(a-1, b-1);
            ss.Sensor::getXY();
            ss.getXY();
        }

        return(0);
    }

```

5.2 Файл Sensor.cpp

Листинг 2 – Sensor.cpp

```

#include "Sensor.h"
#include <iostream>

void Sensor::setX(int x)
{
    this->x = x*3;
}

Sensor::Sensor(int x, int y)
{
    setX(x);
    this->y = y;
}

void Sensor::setXY(int x, int y)
{
    setX(x);
    this->y = y;
}

void Sensor::getXY()
{
    printf("Sensor: %d %d\n", x, y);
}

```

5.3 Файл Sensor.h

Листинг 3 – Sensor.h

```

#ifndef __SENSOR__H

```

```

#define __SENSOR__H

class Sensor
{
private:
    int x;

public:
    int y;

public:
    Sensor(int x, int y);
    void setXY(int x, int y);
    void getXY();

private:
    void setX(int x);
};

#endif

```

5.4 Файл SmartSensor.cpp

Листинг 4 – SmartSensor.cpp

```

#include "SmartSensor.h"
#include <iostream>

SmartSensor::SmartSensor(int x, int y) : Sensor(x, y)
{
    this->x = x;
    this->y = y;
}

void SmartSensor::setXY(int x, int y)
{
    this->x = x;
    this->y = y;
}

void SmartSensor::getXY()
{
    printf("SmartSensor: %d %d\n", x, y);
}

```

5.5 Файл SmartSensor.h

Листинг 5 – SmartSensor.h

```
#ifndef __SMARTSENSOR__H
#define __SMARTSENSOR__H
#include "Sensor.h"

class SmartSensor : public Sensor
{
private:
    int x;

public:
    int y;

public:
    SmartSensor(int x, int y);
    void setXY(int x, int y);
    void getXY();
};

#endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 10.

Таблица 10 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 4 5 2 3	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 SmartSensor: 3 4 Sensor: 3 2	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 SmartSensor: 3 4 Sensor: 3 2
3 4 5 -2 3	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 Sensor: -3 4 SmartSensor: -3 2	Sensor: 9 4 SmartSensor: 3 4 SmartSensor: 3 5 Sensor: 9 5 Sensor: -3 4 SmartSensor: -3 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).